

テクスチャ圧縮基礎知識

GPU圧縮とは？

テクスチャ概論

ゲームを含むPCのすべての画像は、最終的には画面上に表示しなければならない。

そのためにはGPUが処理できなければならず、GPUが処理するデータはすべてVRAM上になくてはならない。

VRAM上に置かれた画像データのことを **テクスチャ Texture** と呼ぶ。

GPUがVRAMからテクスチャを読み取る際、GPUは大量のコアが同時に、好き勝手に画像内のいろんな座標の色データを読もうとする。これを **テクスチャサンプリング** と呼ぶ。

普通の圧縮方法とは？

一般的な画像圧縮フォーマットとしてはjpg, png, 最近の流行りだと webp, avifなどがある。

これらは基本的には、**普通の画像データは、近くのピクセルでは同一の色が連続したりグラデーション状に変化することがほとんどだから、隣や周囲のピクセルからの色の差分だけを記録するようにすればデータがスリムになるはずだ** という発想で作られている。

つまり、**画像データの真ん中あたりの色が急に欲しいと思ったら、データ始点の色とそこからの差分の積み重ねが必要になってしまう** のだ。こういった圧縮方式は、画質をできるだけ保ちつつファイルサイズを限界まで削減しようという発想で作られており、GPU的な突然画像の真ん中の色が欲しくなるという用途には向かない。

ではGPUはどのようにテクスチャをVRAM上に置いているか？というと、基本的にはpng等で圧縮された画像は完全にデコードされ無圧縮でVRAM上に乗る。無圧縮なら突然画像の真ん中の色が欲しくなってもすぐ取り出せる。

GPU対応圧縮フォーマット

ところが無圧縮の画像というのはものすごく容量を食う。RGBAのフルカラーなら2048x2048の画像で16MBだ。これに困ったGPUメーカーは、「GPUの読み出ししかたに対応したテクスチャ圧縮」という技術を生み出した。

ブロック圧縮

基本的な発想は画像を4x4などの正方形ブロックに分割し、各ブロックごとにカラーパレットを作って限られた色数でブロック内の色を表現するというやり方だ。これなら突然画像真ん中の色が欲しくなっても、その座標の色が属するブロックの情報を読むだけでいい。

GPU内でこのアルゴリズムでテクスチャサンプリングをする際は、専用回路が用意されるので展開に要する処理時間はほぼ気にしなくてよい。なんならメモリサイズが小さくなることにより、探索が効率化されるので無圧縮より高速化されることの方が多い。

各種フォーマット

BC(DXT)系/ETC系は、ブロックで切る都合上、画像の解像度が縦も横も4で割り切れる値となっている必要がある。

BC1~BC7 / DXT1~DXT5 / S3TC

むかしむかし、S3という会社が初めて作ったGPU圧縮フォーマット。その後DirectXの標準仕様となり、PC系GPUのスタンダードとなった。**GeForceやRADEONなど、デスクトップ系GPUがWindowsやLinuxで対応する。Intel Macも対応してた。**呼び方が一定しないが、とりあえず **DXT圧縮** と言えばゲーム業界の人間には通じる。

よく使われるのは以下のモード。

モード名	容量(2048x2048)	備考
DXT1(BC1)	2MB	アルファ無し
DXT5(BC3)	4MB	アルファ有り
BC6H	4MB	アルファ無し、HDR対応
BC7	4MB	アルファ有り、高画質

BC6H, BC7は比較的最近追加された高画質モード。しかしエンコードがめちゃ遅い。

ETC2

Ericsson Texture Compressionの略で、最初からスマートフォン向けGPUのテクスチャ圧縮方式として開発された。

初期のAndroidで標準的なテクスチャ圧縮だったが、ETC1ではアルファチャンネルがそもそも無いなど使いにくかった。ETC2でだいぶまともになり、現在は化石のようなスマートフォンでなければ基本的にはETC2に対応しているので **スマートフォンGPUのベースライン** となっている。ETC2はiOSも対応している。

モード名	容量(2048x2048)	備考
ETC2 RGB	2MB	アルファ無し
ETC2 RGBA	4MB	アルファ有り

ASTC

新しめのスマートフォン(2025年現在ならほぼ全てと言っていい)が対応している 新しいGPU圧縮フォーマット。**デスクトップGPUも逆輸入の形で対応しているが、UnityのStandaloneビルドでは利用できない。**(Editorでできるのに...) Adaptive Scalable Texture Compressionの略で、最大の特徴はほかのGPU圧縮が基本的に4x4ブロックで圧縮を行うところ、ブロックサイズを可変としたところにある。

フォーマットが新しいからか非常に高画質で、個人的評価だと4x4なら肉眼ではほとんど差がない。6x6でも充分で、かつ容量は圧倒的に小さくなる。**ぶっちゃけ最強のフォーマット。**欠点はエンコードがやや遅いところ。しかしBC6/7ほどではなく、ランタイムエンコードも実用的な速度と言えるレベル。

モード名	容量(2048x2048)	備考
ASTC 4x4	4MB	ブロックサイズが変わるため、
ASTC 5x5	2.5MB	ASTCの容量は小数点が出る
ASTC 6x6	1.8MB	アルファはすべて対応
ASTC 8x8	1MB	HDRに対応したASTC HDRもある

モード名	容量(2048x2048)	備考
ASTC 10x10	0.6MB	
ASTC 12x12	0.4MB	

8x8や12x12になってくるとさすがに画質は下がるが、例えば2048x2048を1024x1024にリサイズして4x4圧縮するのと、リサイズせずに8x8圧縮するのでは容量が同じで後者のほうが高画質となる。それならブロックサイズ大きくするほうがいいよね。

可変ブロックサイズという仕様のため、画像がブロックサイズで都合よく割り切れないことがほとんどなのでASTCは画像縦横幅の制約がない。
割り切れないのが当たり前なので余りブロックをうまく処理して隠すシステムが備わっている。